CI/CD パイプラインのセキュリティ診断における検知困難な設定ミス検出手法提案

渡邉 大翼[1,*]　島 成佳[1]

概要：現在，CI/CD パイプラインという手法の普及に伴い，運用環境へ引き渡すデプロイ工程での設定ミスが発生している．既存のセキュリティ診断では，「システム」と「データ」の環境差に起因する設定ミス（IAM や RBAC，クラウド関連などの代表的な 5 項目）の検知は困難である．本研究は，ウェブサービスのシステムの設定ミスを検知可能とする検証手法を確立することを目的としており，本番同等のステージング環境上でカナリアテストを実施する手法を提案する．具体的には，「システム」と「データ」の環境差を同時に埋めるために，システム構成を網羅的に検証するステージング環境と，本番ユーザの操作と運用・開発者のリアルタイム監視を連携させたカナリアテストを同時に実施する．検証では AWS 上にテスト用 EC サイトを構築して代表的な 5 項目それぞれ 1 つずつの設定ミスを対して，OWASP ZAP，Nikto のセキュリティ診断ツールでは検知できないことと，提案手法では検知できることを確認した結果について報告する．

キーワード：ウェブサービス，CI/CD パイプライン，デプロイ，設定ミス，セキュリティ診断

A Proposed Method for Detecting Hard-to-Detect Misconfigurations in CI/CD Pipeline Security Assessments

Daisuke Watanabe[1,*]　Shigeyoshi Shima[1]

Abstract: With the widespread adoption of CI/CD pipelines, misconfigurations increasingly occur during the deployment phase to production environments. Existing security diagnostic tools struggle to detect misconfigurations arising from environmental discrepancies between the "system" and the "data" (specifically focusing on five representative categories, including IAM, RBAC, and cloud-related settings). This study aims to establish a verification method capable of detecting such web service misconfigurations by proposing an approach that conducts canary testing within a production-equivalent staging environment. Specifically, to bridge the environmental gap between the system and the data, the proposed method concurrently utilizes a staging environment for comprehensive system configuration verification, and a canary test that links authentic user operations with real-time monitoring by operators and developers. To validate this approach, a mock e-commerce site was constructed on AWS, embedding one intentional misconfiguration for each of the five representative categories. This paper reports the verification results, demonstrating that while conventional security diagnostic tools (OWASP ZAP and Nikto) failed to detect these misconfigurations, the proposed method successfully identified them.

Keywords: Web service, CI/CD pipeline, deployment, misconfigurations, security assessment

1. はじめに

スマートフォンの地図アプリや EC サイト，SNS 等のサービスを構成するウェブサービスは現代社会において必要不可欠なものになっている．これらのウェブサービスは利便性と豊かさを提供する一方で，多様化・高度化するサイバー攻撃という脅威の標的となり，不正アクセスやクレジットカードの不正利用，サービス停止といった深刻な影響を及ぼすセキュリティインシデントが発生している．これらは企業に経済的損失や社会的信用の低下を引き起こす可能性がある．

こうした脅威の深刻さは，公的機関や業界団体の統計からも裏付けられている．日本クレジット協会が 2026 年 3 月に公表した現状報告[1]によると，2025 年におけるクレジットカードの不正利用被害額は 510.5 億円に達している．前年（555.0 億円）からは微減したものの，2014 年の 114.5 億円と比較して約 4.5 倍である．被害額の 93.1%（475.4 億円）が EC サイト等における「クレジットカード番号盗用」によって占められることである．また，警察庁，総務省および経済産業省が令和 7 年 3 月に公表した現状報告[2]によると，令和 6 年における不正アクセス行為の認知件数は 5358 件であった．これは過去最多であった前年（6312 件）からは 15.1%減少しているものの，令和 2 年から令和 4 年までの認知件数が約 1,500～2,800 件台であったのに対し，令和 6 年は 5,358 件であった．同報告書内に示された独立行政法人情報処理推進機構（IPA）への届出状況によると，実際の不正アクセス被害において攻撃対象となった機器の最多は「ウェブサーバ」（57 件）であった．さらに，その侵入原

1 長崎県立大学
University of Nagasaki, 1-1-1 Manabino, Nagayo-cho, Nishisonogi-gun, Nagasaki 851-2195, Japan

* mc125011@sun.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

因として「システムの設定不備」が約 23.4%（33 件）を占め，ソフトウェアの更新不備に次いで多い原因である．ウェブサービスにおけるセキュリティ対策が，社会全体で取り組むべき課題であることを示している．

このような「設定ミスの悪用」という実態は，国際的なセキュリティ基準とも合致している．Open Worldwide Application Security Project(OWASP)が発表した「OWASP TOP 10:2025」[3]においても，「A02:2025 セキュリティ設定の不備（Security Misconfiguration）が掲載されている．ここでのセキュリティ設定の誤りとは，システム，アプリケーション，またはクラウドサービスが安全でない状態で構築され，攻撃者に悪用され得る脆弱性が生じている状態を指す．

近年のシステム環境の高度化や複雑化に伴い，開発者がすべての構成要素や設定を把握することは困難となっている．また，開発プロセスにおいて，ウェブサービスを開発環境から運用（本番）環境へ引き渡すデプロイ工程では，環境固有の設定変更が発生する．

本論文では，開発環境から運用環境への移行時に発生する設定ミスに着目し，セキュリティ診断（ウェブアプリケーション診断，プラットフォーム診断）で検知困難な設定ミスを 5 項目に整理し，開発環境と運用環境の環境差を埋める手法として，本番同等のステージング環境上で，システム構成を確認し，本番ユーザの操作と運用・開発者のリアルタイム監視を連携させたカナリアテストを同時に実施する手法を提案する．さらに，AWS（Amazon Web Services）を用いた実環境における実証実験を通じて，提案手法によって対象とする設定ミスを識別可能であるかを実証実験により評価する．

本論文の構成は以下の通りである．2 節では研究の流れと本研究の位置づけ，3 節では検証環境の構築，4 節では設定ミスの組み込み, 5 節では検証と結果, 6 節では考察，7 節ではまとめについて述べる．

2. 研究の流れと本研究の位置づけ

本研究は，デプロイ工程で発生する設定ミスを検知し，運用環境への設定ミスの混入を防止する手法を確立することを目指している．本研究の全体の流れを以下に示す．

1. 設定ミスの発生状況の調査
   開発者へのインタビュー調査を実施し，CI/CD パイプラインの各工程における設定ミスの発生状況を調査する．
2. 既存のセキュリティ診断で検知が困難な設定ミスの抽出
   ウェブアプリケーション診断およびプラットフォーム診断を対象として，既存のセキュリティ診断では検知が困難な設定ミスを整理し，代表的な 5 項目を検知対象として定義する．
3. 開発環境と運用環境の差の整理
   開発環境と運用環境の間に生じる環境差を，「システムの差」と「データの差」に分類する．
4. 設定ミス検知手法の概念設計
   「システムの差」を補完するステージング環境と，「データの差」を補完するカナリアテストを組み合わせた設定ミス検知手法を概念設計する．
5. 実証環境の構築と設定ミスの組み込み
   AWS 上に実証環境を構築し，2 で定義した 5 項目の設定ミスを意図的に組み込む．
6. 既存のセキュリティ診断による検知可否の確認
   構築した実証環境に対して既存のセキュリティ診断を実施し，5 項目の設定ミスに対する直接的な検知可否を確認する．
7. 提案手法の実装および検証
   4 で概念設計した提案手法を実証環境に適用し，ステージング環境によるシステム構成および設定状態の確認と，カナリアテストによるユーザ操作に伴う挙動の観測を実施する．
8. 提案手法の評価
   7 で得られた検証結果をセキュリティ挙動チェックリストに定義した期待状態と照合し，5 項目の設定ミスに対する提案手法の検知可能性を評価する．

以上の 1〜8 のうち, 1〜4 は文献[4]で実施した内容であり，5〜8 は本論文で新たに実施する内容である．文献[4]では，設定ミスの発生状況の調査から提案手法の概念設計までを行っている．一方，本論文では，文献[4]で概念設計した提案手法を実証環境に適用し，既存のセキュリティ診断との比較を含めて検知可能性を評価する．

2.1 5 項目の代表的な設定ミス

本研究では，文献[4]で整理された，既存のセキュリティ診断では検知が困難な設定ミスを対象として，提案手法の検知可能性を評価する．また，ウェブアプリケーション診断およびプラットフォーム診断を対象として，既存のセキュリティ診断では検知が困難な設定ミスを整理している．本研究では，その中から以下の 5 項目を代表的な設定ミスとして対象とする．

①RBAC（Role-Based Access Control）の設定ミス：
特定のユーザや部門に限定するなど，システム固有の業務仕様に基づく権限設定の妥当性は，診断ツールによる自動判定が困難である．

②アプリケーションログの不適切な設定：
ログに記録される内容の適切性は，外部からの通信を対象とした診断では確認が困難である．

③バックエンド API の認証・認可ミス：
内部利用を想定した API など，診断対象として把握されていないエンドポイントの認証・認可ミスは検知が困難である．

④クラウドストレージの設定ミス：
ストレージが公開状態であってもアプリケーション自

© 2026 Information Processing Society of Japan

体は正常に動作する場合があり，さらに保存データの機密性まで診断ツールが判断することは困難である．

⑤IAM（Identity and Access Management）の設定ミス：
IAM の権限設定が過剰であっても，アプリケーションが正常に動作する場合があり，通常の機能確認では検知が困難である．

2.2 提案手法の概念設計

文献[4]では，CI/CD パイプラインのデプロイ工程での開発環境と運用環境の間に生じる環境差を「システムの差」と「データの差」に分類し，これらを補完するために，ステージング環境とカナリアテストを組み合わせた設定ミス検知手法が概念設計されている．

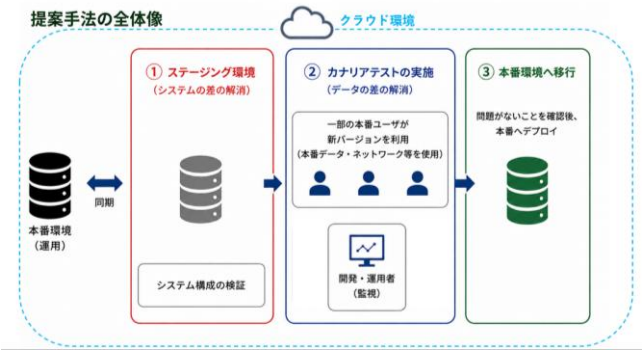


図 1 提案手法のイメージ図

Figure 1 Proposed Approach Conceptual Diagram

ここで，「システムの差」とは，開発環境と運用環境におけるサーバ，API，クラウドリソース，権限設定等のシステム構成や設定状態の差を指す．一方，「データの差」とは，実利用時のユーザ操作や入力データ等によって生じる実行時の状態の差を指す．図 1 が示すように提案手法では，ステージング環境においてシステム構成および設定状態を確認することで「システムの差」を検証し，カナリアテストにおいてユーザ操作に伴う実際の挙動を観測することで「データの差」に起因する影響を検証する．これらを組み合わせることで，開発環境では顕在化しにくく，運用環境への移行後に発現する設定ミスの検知を目指す．

本研究では，この概念設計に基づき，ステージング環境による構成確認とカナリアテストによる挙動観測を実証環境に適用する．ステージング環境では，権限設定，API，クラウドリソース等の構成上の状態を確認する．一方，カナリアテストでは，ユーザ操作に伴って実際に現れる画面上の挙動やシステム内部の処理結果を観測する．これらの確認結果をセキュリティ挙動チェックリストに定義した期待状態と照合することで，システムの設定状態とユーザ操作に伴う実際の挙動を対応付け，設定ミスを検知する．

また，本研究では，ステージング環境における構成確認およびカナリアテストによる挙動観測の判定基準として，セキュリティ挙動チェックリストを策定する．本チェックリストは，対象システムの機能要件，システム構成，環境ごとの各リソースの設定，および 5 項目の設定ミスを基に，ユーザ操作とシステム内部の処理および設定を対応付け，本来期待されるセキュリティ挙動とその判定条件を定義する．チェックリストは，以下の 4 段階で策定する．

(1)機能要件とシステム構成の対応付け
対象システムの機能要件と図 3 のアーキテクチャ構成を整理し，各ユーザ操作が利用する API，サーバ処理，AWS リソースおよび権限を対応付ける．

(2)監視対象の抽出
通常の機能確認で把握できる項目と，システム内部のセキュリティ挙動を確認する必要がある項目を区分する．

(3)期待状態の定義
各ユーザ操作について，許可される権限，実行される API，アクセス可能なデータ，出力されるログおよび利用される AWS リソース等，本来期待されるセキュリティ状態を定義する．

(4)判定条件の定義
期待状態から逸脱した場合に設定ミスと判定できる事象を抽出し，観測項目および判定条件として整理する

3. 検証環境の構築

本研究では，前節で示した提案手法を実環境に適用するため，AWS（Amazon Web Services）を用いた実証環境を構築した．Flexera が実施した「2026 State of the Cloud Report」では，753 のクラウド利用企業・意思決定者を対象とした調査において，現在 AWS 上で一部または主要なワークロードを実行している組織の割合は 83%であり，Azure の 79%を上回っている[5]．また，AWS では IAM，EC2，S3 等のサービスを利用でき，本研究で対象とする 5 項目の設定ミスに関係する構成要素を同一環境上で構築できる．以上から，本研究では実証環境として AWS を採用した．

3.1 実証実験の前提とスコープ

本実証実験では，文献[4]で概念設計された提案手法を前提として，開発環境と運用環境の間に生じる環境差を検証対象とする．クラウド上に「システムの差」を埋めるステージング環境によるシステム構成の検証と，「データの差」を埋めるカナリアテストによる実利用時の検証を組み合わせた検証プロセスを概念設計した．その概要を図 2 に示す．

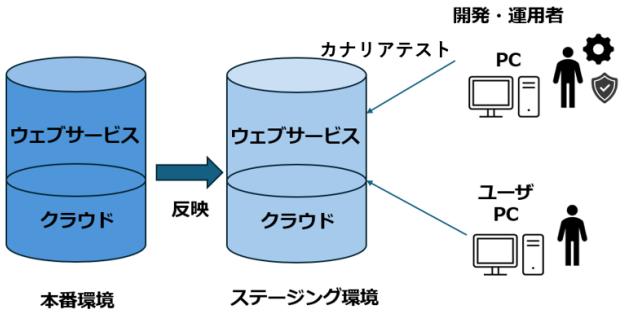


図 2 想定するウェブサービス構成のイメージ図[4]

© 2026 Information Processing Society of Japan

Figure 2　Conceptual Diagram of the Assumed Web Service Configuration[4]

3.2 検証用ウェブサイトの構築

本研究では，AWS 上に Amazon Linux 2023 を搭載した EC2 インスタンスを構築し，Apache HTTP Server を用いて簡易的な EC サイトを構築した．実証実験環境の全体構成を図 3 に示す．

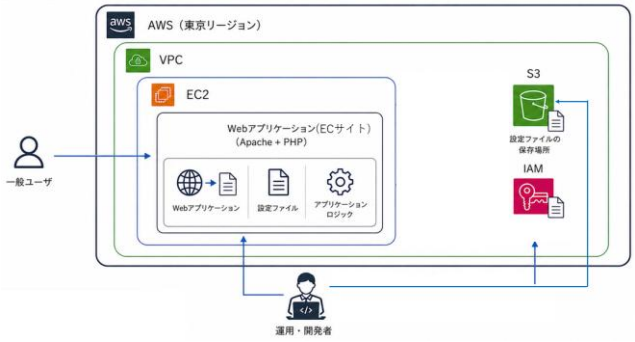


図 3　AWS を用いた実証実験環境の構成

Figure 3　Configuration of the Experimental Environment Using AWS

システムは AWS 東京リージョン内の VPC 上に構築され，EC2 上で Web アプリケーションが稼働している。設定ファイルは S3 に保存され，IAM によって各 AWS リソースへのアクセス権限が管理される．一般ユーザは Web アプリケーションを利用し，運用・開発者はシステムの設定や管理を行う。

Web サイトは Apache のドキュメントルートである /var/www/html に配置し，外部から HTTP によるアクセスを可能とした．本環境を基盤として，4 節では 5 項目の設定ミスを組み込み，5 節では既存のセキュリティ診断とカナリアテスト，提案手法での検知可否を検証する．

4.　5 項目の代表的な設定ミスの組み込み

本節では，3 節で構築した実証環境（図 3）に対して，文献[4]で対象とした 5 項目の設定ミスを意図的に組み込み，各設定ミスが実証環境上で成立していることを確認する．各設定ミスについて，仕様と実装対象，実装方法，確認方法と結果の 3 つの観点から示す．

①ロールベースアクセス制御（RBAC）の設定ミス

・仕様と実装対象：

RBAC（Role-Based Access Control）は，利用者に割り当てられたロールに基づいてアクセス権限を制御する方式である．本実証環境では，図 3 に示す EC2 上の Web アプリケーションを対象とし，root，admin，test の 3 種類のロールを設定した．また，EC サイトの操作として，①商品追加，②価格設定，③商品削除，④在庫確認，⑤購入，⑥権限管理の 6 種類を定義し，一般顧客を想定した test には⑤購入のみを許可する仕様とする．

・実装方法：

検証環境を簡易化するため，フロントエンド（index.html）のスクリプトに各ロールと UI 表示を対応付ける権限マッピングを実装する．設定ミスとして，test ロールに本来許可されていない③商品削除の権限を付与する．

・確認方法と結果：

test ユーザでログインし，商品削除機能が表示され，実行可能であることを確認（図 4）．

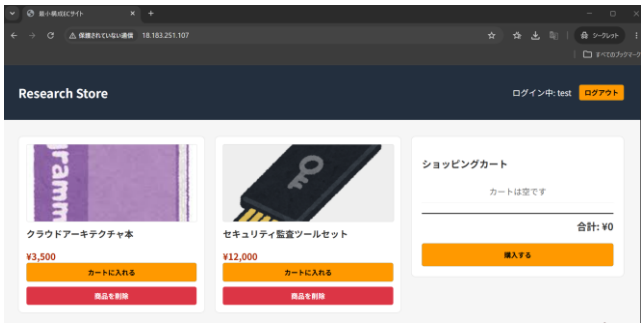


図 4　RBAC 設定ミスの検証結果

Figure 4　RBAC Misconfiguration

②アプリケーションログの不適切な設定

・仕様と実装対象：

アプリケーションログは，運用時の監視や障害解析に利用される．本実証環境では，図 3 に示す EC2 上の Web アプリケーションを対象とし，ログイン処理において認証情報をログへ出力しないことを本来の仕様とする．

・実装方法：

設定ミスとして，フロントエンド（index.html）の login() 関数にデバッグ用のログ出力処理を追加し，登録ユーザの認証情報をブラウザのコンソールへ出力する状態とする．

・確認方法と結果：

test ユーザでログインし，認証情報がブラウザのコンソール上に出力されることを確認（図 5）．

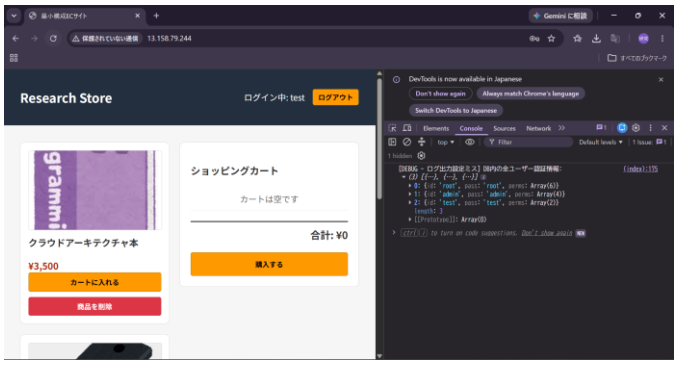


図 5　アプリケーションログ設定ミスの検証結果

Figure 5　Log Misconfiguration

③バックエンド API の認証・認可ミス

・仕様と実装対象：

バックエンド API では，リクエスト送信者の認証および権限を確認し，許可された利用者のみが処理を実行でき

© 2026 Information Processing Society of Japan

ることが求められる．本実証環境では，図 3 に示す EC2 上の Web アプリケーションにおける価格変更 API を対象とし，未認証状態から価格変更処理を実行できないことを本来の仕様とする．

・実装方法：

設定ミスとして，EC2 上に配置した価格変更 API（/var/www/html/api/update_price_.php）からセッションおよび Cookie による認証・認可の検証処理を除外し，受信した POST パラメータをそのまま処理する実装する．

・確認方法と結果：

未認証状態から API へリクエストを送信した結果，リクエストが拒否されず，HTTP 200 OK が返され，処理が受理されることを確認（図 6）．

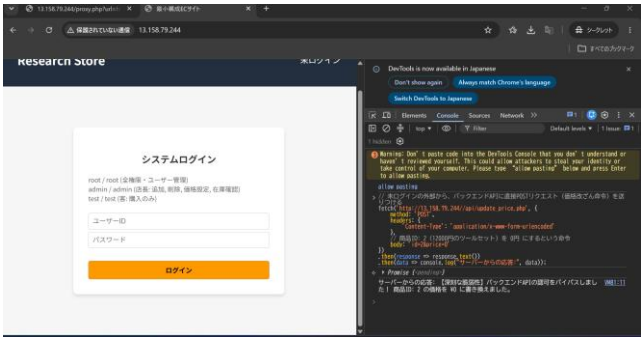


図 6　API 認証・認可設定ミスの検証結果

Figure 6　API Authentication and Authorization Misconfiguration

④クラウド環境でのストレージ設定ミス

・仕様と実装対象：

Amazon S3 では，保存データの用途に応じてアクセス範囲を設定し，非公開データを未認証の利用者から保護する必要がある．本実証環境では，図 3 右上に示す Amazon S3 を対象とし，非公開を想定したデータを未認証利用者から取得できないことを本来の仕様とする．

・実装方法：

設定ミスとして，対象 S3 バケットの「パブリックアクセスをすべてブロック」を無効化し，オブジェクトの読み取りを許可するバケットポリシーを適用する．さらに，商品画像とともに，非公開を想定した customer_list_2026.csv を配置する．

・確認方法と結果：

「test」ユーザでログインし，商品画像が S3 から正常に読み込まれることを確認．次に，未認証状態から開発者ツールを用いて HTML ソースを確認し，商品画像が読み込まれている S3 バケットのベース URL を特定（図 7）．

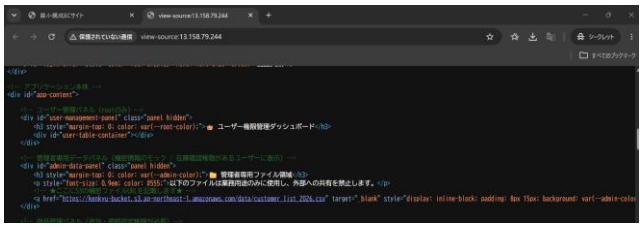


図 7　S3 バケット URL の特定

Figure 7　S3 Bucket URL Identification

その URL へアクセスした結果，本来非公開であるべき customer_list_2026.csv を取得可能であることを確認（図 8）．

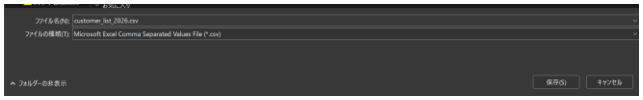


図 8　S3 機密ファイルへの不正アクセス

Figure 8　Unauthorized Access to S3 Sensitive Files

⑤IAM の設定ミス

・仕様と実装対象：

IAM（Identity and Access Management）は，AWS リソースに対するアクセス権限を管理するサービスである．本実証環境では，図 3 に示す EC2 インスタンスに関連付けられた IAM ロールを対象とし，アプリケーションの動作に必要な範囲に権限を限定することを本来の仕様とする．

・実装方法：

設定ミスとして，AmazonS3FullAccess ポリシーを持つ IAM ロール「kenkyuEC2overrole」を作成し，検証用 EC2 インスタンスに付与する．さらに，この IAM 設定ミスが外部から利用可能となる条件を再現するため，EC2 上に検証用プロキシ（proxy.php）を配置し，外部から受け取った URL を検証せずサーバ側からリクエストする処理を実装する．

・確認方法と結果：

未認証状態からプロキシを介して EC2 インスタンスメタデータサービスへアクセスした結果，IAM ロール名および対応する一時認証情報を取得可能であることを確認（図 9，図 10）．

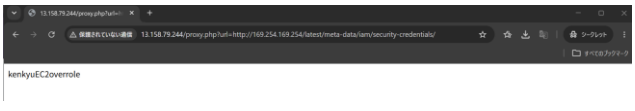


図 9　IAM ロール名の取得

Figure 9　IAM Role Name Retrieval



図 10　IAM 一時認証情報の取得

Figure 10　IAM Temporary Credential Retrieval

© 2026 Information Processing Society of Japan

5. 検証結果

本節では，4 節で構築した実証環境を対象として，既存のセキュリティ診断，カナリアテストによるリアルタイム監視，およびセキュリティ挙動チェックリストを用いた提案手法による検証結果を示す．なお，ステージング環境単体は提案手法を構成する一要素であるため，独立した比較対象とはせず，既存のセキュリティ診断およびカナリアテストによるリアルタイム監視と，これらを組み合わせた提案手法を比較対象とする．

セキュリティ診断には，診断対象に応じて複数の手法が存在する．その中でも，ウェブアプリケーションに対する診断と，Web サーバ等を対象とする診断は，ウェブサービスのセキュリティ確認に用いられる代表的な診断手法である．そこで本研究では，既存のセキュリティ診断による直接的な検知可否を確認するため，ウェブアプリケーション診断ツールとして OWASP ZAP[6]，Web サーバ等を対象とするプラットフォーム診断ツールとして Nikto[7]を用いる．これら 2 種類の診断により，4 節で組み込んだ 5 項目の設定ミスに対する直接的な検知可否を確認する．

次に，同一の実証環境において，一般ユーザ相当の利用者による操作と運用・開発者によるリアルタイム監視を実施する．さらに，ステージング環境におけるシステム構成および設定状態を確認し，これらの結果をセキュリティ挙動チェックリストに定義した期待状態と照合することで，提案手法による設定ミスの検知可否を評価する．

5.1 OWASP ZAP での検証

OWASP ZAP を起動し，検証対象である EC2 インスタンスのパブリック IP アドレスをスキャン対象として設定し，診断を実行する．

図 11 に示すように，診断の結果，セキュリティヘッダの欠如，サーバヘッダを通じたバージョン情報の露出，HTML 内の文字列に反応した「疑わしいコメント」等が検知される．これらは，実証環境の構築時に意図せず残存した設定や Web アプリケーション内の記述に起因する検知結果であり，4 節で意図的に組み込んだ 5 項目の設定ミスには該当しない．

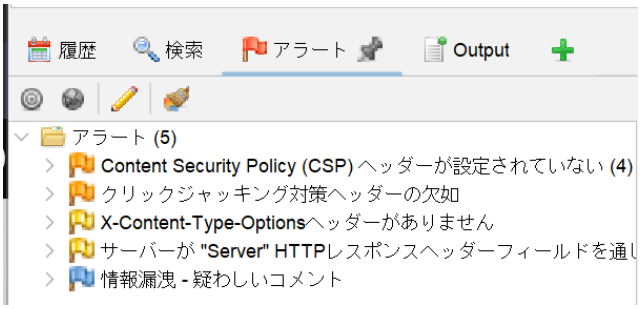


図 11　OWASP ZAP による診断結果

Figure 11　OWASP ZAP Scan Results

一方，4 節で組み込んだ①～⑤の 5 項目の設定ミスについて，検知結果を確認できない．

5.2 Nikto での検証

ローカル端末上の Docker 環境（Kali Linux コンテナ）から Nikto（v2.6.0）を実行し，検証用 EC2 インスタンスを対象としてスキャンを実施する．また，EC2 インスタンスへ SSH 接続し，Apache のアクセスログ（/var/log/httpd/access_log）を tail -f コマンドでリアルタイムに監視することで，スキャン時に発生する HTTP リクエストを確認する．

図 12 に示すように，約 8,200 件の HTTP リクエストを発生させたスキャンの結果，HTTP TRACE メソッドの有効化，Directory Indexing の有効化，Apache および PHP のバージョン情報の露出等が検知される．これらも，実証環境の構築時に意図せず残存した Web サーバの設定等に起因する検知であり，4 節で意図的に組み込んだ 5 項目の設定ミスには該当しない．

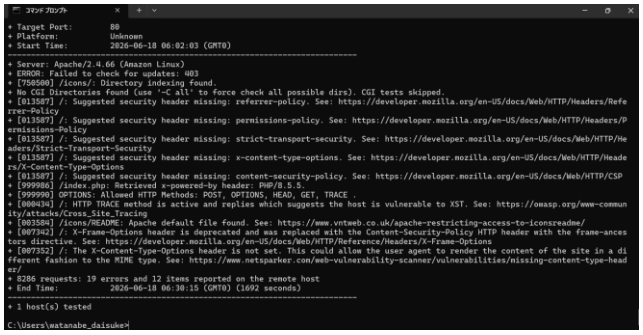


図 12　Nikto による診断結果

Figure 12　Nikto Scan Results

一方，Nikto についても，4 節で組み込んだ①～⑤の 5 項目の設定ミスの検知結果を確認できない．

5.3 カナリアテストでの検証

4 節で構築した実証環境に対して，一般ユーザ相当の利用者による操作と運用・開発者によるリアルタイム監視を実施する．本検証では，ユーザ操作を起点として発生するシステムの挙動を観測し，対象とする 5 項目の設定ミスについて，ユーザ操作や画面上の挙動から識別可能なものを確認する．

本実証実験では，一般ユーザ相当の利用者 1 名が実証環境を操作する様子を画面共有し，運用・開発者（著者）がリアルタイムに監視する方法を用いる．一般ユーザ相当の利用者は，test ユーザによるログイン，商品閲覧，購入等の通常操作を行う．その結果以下の設定ミスを検知できる．

①RBAC の設定ミス：

test ユーザでログインした際に，本来許可されていない商品削除機能が画面上に表示され，削除可能となることを確認できる（図 13）．

© 2026 Information Processing Society of Japan

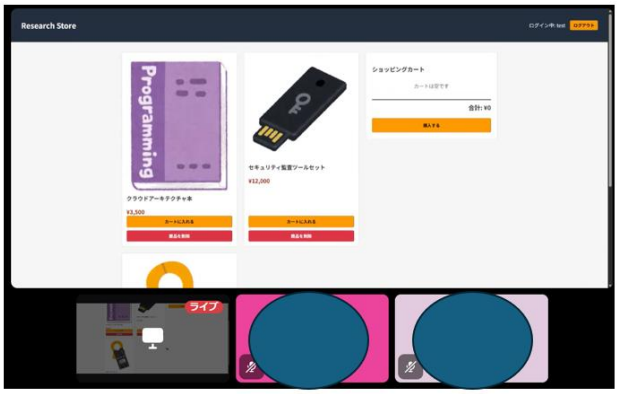


図 13　リアルタイム監視の様子

Figure 13　Real time monitoring in progress

一方，①～④の 4 項目の設定ミスはユーザ操作や画面上の挙動だけから設定ミスを直接識別することは困難である．

5.4 提案手法による検証

2.2 節の手順で作成したチェックリストは，図 3 に示す AWS 実証環境およびそこに構築した Web アプリケーションを対象としたものであり，他のシステムや異なるアーキテクチャへそのまま適用することを想定したものではない．

策定したセキュリティ挙動チェックリストを用いて，システム構成および設定状態とユーザ操作に伴う挙動を確認する．その結果，以下の 5 項目について設定ミスを識別できる．

①RBAC：test ロールに商品削除権限が付与されていることを構成上確認するとともに，test ユーザでログインした際に商品削除機能が表示されることを確認する．これにより，設定上の逸脱と実際の挙動を対応付ける．

②アプリケーションログ：認証情報を出力するデバッグ処理の存在を確認するとともに，ログイン操作に伴って認証情報が出力されることを確認する．

③バックエンド API：認証・認可処理が存在しないことを確認するとともに，未認証状態からのリクエストが受理されることを確認する．

④S3：公開設定およびバケットポリシーを確認するとともに，未認証状態から非公開ファイルを取得可能であることを確認する．

⑤IAM：過剰な権限を持つ IAM ロールが EC2 に付与されていることを確認するとともに，そのロールに対応する一時認証情報を取得可能となる状態を確認する．

以上より，システム構成および設定状態とユーザ操作に伴う挙動を対応付けることで，対象とした 5 項目すべてについて，設定上の逸脱とセキュリティ上の挙動を関連付けて検知できる．

5.5 検証結果のまとめ

表 1 に各手法による 5 項目の設定ミスの検知結果を示す．

表 1　各手法による設定項目の検知表

Table 1　Detection table of settings by each method

設定ミス	5.1 ZAP	5.2 Nikto	5.3 カナリアテスト	5.4 提案手法
① RBAC	×	×	○	○
② アプリケーションログ	×	×	×	○
③ API認証・認可	×	×	×	○
④ S3	×	×	×	○
⑤ IAM	×	×	×	○

診断ツールである OWASP　ZAP，Nikto では，5 項目の設定ミスを 1 つも検知できない．カナリアテストによるリアルタイム監視では，①RBAC の設定ミスのみ検知できる．提案手法では，①～⑤の設定ミスを検知できる．

6. 考察

本節では，実証実験の結果を踏まえ，実証環境と適用可能性，セキュリティ挙動チェックリスト，提案手法の有効性と今後の発展について考察する．

6.1 実証環境と適用可能性

本研究では，開発環境から運用環境への移行時に発生する設定ミスを対象として，ステージング環境によるシステム構成および設定状態の確認と，カナリアテストによるユーザ操作に伴う挙動観測を組み合わせた提案手法を AWS 上の実証環境に適用する．その結果，既存のセキュリティ診断では検知されなかった 5 項目の設定ミスについて，提案手法によって設定上の逸脱と実際のセキュリティ上の挙動を対応付けて検知できることを確認できる．

本実証実験では，提案手法の基本的な検知メカニズムを確認するため，AWS 上に構築した Web アプリケーションを対象として検証を実施する．カナリアテストでは，一般ユーザ相当の利用者による操作を運用・開発者がリアルタイムに監視することで，ユーザ操作に伴うシステムの挙動とシステム構成および設定状態を対応付けて確認した．この検証により，提案手法における「システムの差」と「データの差」をそれぞれ確認するという基本的な考え方を実証できる．

今後は，ユーザ操作に伴う API 通信，ログ出力，権限行使，クラウドリソースへのアクセス等の詳細なシステム挙動を自動的に取得し，セキュリティ挙動チェックリストおよびシステム構成との対応付けを行うことで，より効率的な検知へ発展させることができると考えられる．また，複数のユーザによる操作や多様な入力データを含む，より実運用に近い環境で検証を行うことで，提案手法による検知可能性をさらに評価できると考えられる．

また，提案手法の基本的な考え方は，特定の AWS サー

© 2026 Information Processing Society of Japan

ビスそのものに依存するものではないため，他のクラウド環境やオンプレミス環境にも適用できる可能性がある．ただし，本研究で作成したセキュリティ挙動チェックリストは，図 3 に示す AWS 上の実証環境を前提としているため，適用対象を変更する場合には，対象環境のアクセス制御，ストレージ，ログ，監視機構等に応じて，期待状態および判定条件を再定義する必要がある．

6.2 セキュリティ挙動チェックリスト

本研究では，対象システムの機能要件，システム構成，各リソースの設定および 5 項目の設定ミスを基に，ユーザ操作とシステム内部の処理および設定を対応付けたセキュリティ挙動チェックリストを作成する．チェックリストを用いることで，単に機能が正常に動作するかを確認するだけではなく，その操作がどの権限で実行され，どの API や AWS リソースが利用され，どのようなログが出力されることが期待されるかといったセキュリティ上の期待状態を明確化できる．

実証実験では，このチェックリストを用いてシステム構成とユーザ操作に伴う実際の挙動を対応付けることで，5 項目すべてについて設定上の逸脱を検知できる．このことから，セキュリティ挙動チェックリストは，構成情報と実際の動作を結び付けるための判定基準として有効であると考えられる．

一方，本研究で作成したチェックリストは図 3 に示す実証環境を対象として作成したものであり，他のシステムへそのまま適用することはできない．しかし，機能要件とシステム構成を対応付け，期待されるセキュリティ挙動とその判定条件を定義するという作成方法は，他のシステムにも適用可能である．今後は，システムの仕様書，構成情報およびリソース設定からチェック項目を体系的に生成する方法を検討することで，異なるシステムへの適用性を高められると考えられる．

6.3 提案手法の有効性と課題

本研究では，OWASP ZAP および Nikto を用いた既存のセキュリティ診断，カナリアテストによるリアルタイム監視，およびステージング環境での構成確認とカナリアテストを組み合わせた提案手法について，5 項目の設定ミスに対する検知結果を比較する．その結果，OWASP ZAP および Nikto では 5 項目の設定ミスを検知できなかったのに対し，カナリアテストによるリアルタイム監視では①RBAC の設定ミスを検知でき，提案手法では 5 項目すべてを検知できる．

この結果から，提案手法の強みは，既存のセキュリティ診断による外部からの診断だけでは確認が困難な設定ミスに対して，システム構成および設定状態とユーザ操作に伴う実際の挙動を対応付けて確認できる点にあると考えられる．一方，提案手法では，ステージング環境におけるシステム構成および設定状態の確認，カナリアテストによるリアルタイム監視，ならびにこれらの結果を対応付けるためのセキュリティ挙動チェックリストの策定が必要であり，既存のセキュリティ診断と比較して，人的・運用上のコストが大きくなる点が課題である．

6.4 セキュリティ診断で検知困難な設定ミス

本研究では，既存のセキュリティ診断では検知が困難な設定ミスとして代表的な 5 項目を対象とし，提案手法による検知可能性を検証する．今後は，開発環境から運用環境への移行時に発生する設定ミスについてさらに調査を進め，今回対象とした 5 項目以外にも，既存のセキュリティ診断では検知が困難な設定ミスを抽出する必要がある．

さらに，抽出した設定ミスについても提案手法を適用し，ステージング環境による構成確認およびカナリアテストによる挙動観測によって検知可能であるかを検証することで，提案手法の検知対象を拡張できると考えられる．対象となる設定ミスの種類を段階的に増やし，より多様な設定ミスを含む環境で検証を行うことで，提案手法の適用範囲および検知可能性をより明確にできると考えられる．

7. まとめ

本研究では，既存のセキュリティ診断では直接的な検知が困難な 5 項目の設定ミスを対象とし，AWS 上に実証環境を構築して検知手法を検証した．OWASP ZAP および Nikto では対象の設定ミスは直接検知されなかった一方，ステージング環境の構成確認とユーザ操作時の挙動観測をセキュリティ挙動チェックリストと照合することで，5 項目の設定ミスを検知できることを確認した．今後は，ユーザ操作に伴う詳細なシステム挙動の自動取得と対応付けを実装し，より実環境に近い条件で提案手法の有効性を評価する．

参考文献

[1] 一般社団法人日本クレジット協会：クレジットカード不正利用被害の発生状況 四半期：主要事業者ベース，日本クレジット協会（オンライン），入手先〈https://www.j-credit.or.jp/information/statistics/download/toukei_03_g.pdf〉(参照 2026-08-06)．

[2] 警察庁：不正アクセス行為の発生状況及びアクセス制御機能に関する技術の研究開発の状況，警察庁（オンライン），入手先〈https://www.npa.go.jp/bureau/cyber/pdf/R070313_access.pdf〉，（参照 2026-08-06)．

[3] OWASP：OWASP Top 10:2025，OWASP（オンライン），入手先〈https://owasp.org/Top10/2025/〉，(参照 2026-08-06)．

[4] 渡邉大翼，島成佳.CI/CD パイプラインのセキュリティ診断における検知困難な設定ミス対策提案.第 110 回 CSEC・第 60 回 SPT 合同研究発表会（セキュリティーサマーサミット 2025）

[5] Flexera：2026 State of the Cloud Report，Flexera（オンライン）,入手先〈https://info.flexera.com/CM-REPORT-State-of-the-Cloud〉（参照 2026-08-19)．

[6] ZAP Dev Team：Zed Attack Proxy (ZAP)，ZAP（オンライン），入手先〈https://www.zaproxy.org/ 〉（参照 2026-08-20)．

[7] CIRT.net：Nikto，CIRT.net（オンライン），入手先〈https://cirt.net/nikto/ 〉（参照 2026-08-20)．

© 2026 Information Processing Society of Japan